

Inyección de dependencias por Constructor frente a DI por campo

En frameworks como **Spring** y **Quarkus**, la **inyección de dependencias (Dependency Injection o DI)** es una técnica ampliamente utilizada para gestionar la creación de objetos y la resolución de sus dependencias. Estos frameworks usan anotaciones como `@Autowired` (en Spring) y `@Inject` (una anotación estándar de Java para DI) para inyectar dependencias automáticamente en las clases, ya sea a través de **constructor**, **setter** o **campos (fields)**. Sin embargo, la **inyección por campo** suele ser desaconsejada, y aquí te explico por qué:

Ejemplo: Comparación entre Inyección por Campo e Inyección por Constructor

Imaginemos una clase de servicio `OrderService` que necesita un repositorio `OrderRepository` para gestionar las órdenes de compra. Este servicio podría inyectarse por campo o por constructor:

Inyección por Campo (desaconsejada):

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service
public class OrderService {

    @Autowired
    private OrderRepository orderRepository;

    public void placeOrder(Order order) {
        if (orderRepository != null) {
            orderRepository.save(order);
        }
    }
}
```

Con este enfoque, `orderRepository` es inyectado **después** de la creación de `OrderService`. Si se llama al método `placeOrder()` antes de que Spring inyecte `orderRepository`, la clase lanzará un `NullPointerException`.

Inyección por Constructor (recomendada):

```
import org.springframework.stereotype.Service;

@Service
public class OrderService {

    private final OrderRepository orderRepository;

    public OrderService(OrderRepository orderRepository) {
        this.orderRepository = orderRepository;
    }
}
```

```
    }

    public void placeOrder(Order order) {
        orderRepository.save(order);
    }
}
```

Aquí, `orderRepository` es pasado como parámetro al constructor de `OrderService`, garantizando que siempre esté disponible al momento de la creación. Además, al ser declarado como `final`, este campo se vuelve inmutable, mejorando la seguridad y la predictibilidad del código.

1. Riesgo de Estado Inválido

- Con la inyección por campo, las dependencias se inyectan **después de que el objeto ha sido creado**, lo que significa que el objeto puede existir sin que las dependencias necesarias estén listas. Esto puede llevar a que el objeto esté en un **estado inválido** hasta que los campos sean inyectados por el framework.
- Por ejemplo, si se llama a un método antes de que las dependencias sean inyectadas, se pueden producir errores como `NullPointerException`.

2. Dificultad para Realizar Pruebas

- La inyección por campo **complica las pruebas** porque las dependencias no son explícitas en el constructor, lo que dificulta reemplazarlas en los tests unitarios.
- Con la **inyección por constructor**, las dependencias son provistas explícitamente, permitiendo pasar fácilmente mocks o implementaciones alternativas al constructor durante las pruebas. Esto simplifica la configuración y permite un mejor control sobre las condiciones de prueba.

3. Dependencias No Explícitas

- La inyección por campo oculta las dependencias, por lo que al instanciar una clase, no se sabe qué dependencias necesita a menos que revise sus campos.
- Por el contrario, con **inyección por constructor**, todas las dependencias están listadas en la firma del constructor, dejando claro qué se requiere para crear una instancia de la clase. Esta transparencia mejora la **legibilidad y mantenibilidad del código**, especialmente en proyectos grandes.

4. Inmutabilidad y Campos `final`

- Con la inyección por campo, las dependencias no pueden ser **declaradas como `final`** porque no se inicializan en el momento de la creación del objeto. El uso de `final` garantiza que un campo, una vez configurado, no se puede modificar, lo que apoya la **inmutabilidad**.
- La inyección por constructor permite que las dependencias se declaren como `final`, haciéndolas inmutables y asegurando que se establezcan una sola vez sin riesgo de cambios acci-

denciales. Las dependencias inmutables contribuyen a tener un código más **seguro y predecible**.

5. Limitaciones en la Lógica de Inicialización

- Dado que la inyección por campo inyecta las dependencias **después de que el constructor ha sido ejecutado**, cualquier lógica de inicialización que dependa de esas dependencias **no puede realizarse inline o en el constructor**. Esto puede llevar a un código más complejo, ya que la inicialización debe postergarse hasta que la inyección esté completa.
- La inyección por constructor no tiene esta limitación, ya que las dependencias se proporcionan inmediatamente al instanciar el objeto, permitiendo usarlas en la lógica de inicialización dentro del propio constructor.

Por Qué se Prefiere la Inyección por Constructor

La inyección por constructor aborda cada uno de estos problemas al asegurar que:

- Las dependencias estén disponibles inmediatamente cuando el objeto es creado.
- Las dependencias sean explícitas en el constructor, lo cual mejora la legibilidad y simplifica las pruebas.
- Los campos puedan declararse como `final`, lo que aumenta la inmutabilidad y la predictibilidad del código.

Seguir la práctica recomendada de **inyección por constructor** permite tener un código más robusto, fácil de mantener y de probar, lo cual es por lo que frameworks como Spring recomiendan este enfoque sobre la inyección por campo.